

make sure the method you want to call actually exists. Even better, if the method doesn't exist, you'll get a compile error: Xcode will refuse to build your app, thus banishing to oblivion another possible source of bugs. The compiler will refuse to compile and instead throw back the error "Use of unresolved identifier 'addNewFireflyReference'".

4. BUILT-IN TUPLE COMPARISON:

For tuples with upto six elements, Swift 2.2 onwards allows tuples to be compared with each other without the use of complicated syntax that compares each element. For example, the code snippet provided here is sufficient to compare tuples and will report true if all elements match. The code will print "No match." in this case. However, a fair word of warning: the tuple will be equal even if the element labels don't match. So for the given snippet, even if the element "last" is called "middle", Swift will still consider them to be equal.



```
let person1 = (first: "Meghana", last: "Gupta")
let person2 = (first: "Varun", last: "Gupta")

if person1 == person2 {
    print("They match. They're the same person.")
} else {
    print("No match.")
}
```

5. DEPRECATION OF C-STYLE LOOPS:

For those accustomed to the C style of writing for loops, Swift 2.2 will come as a bit of a rude shock. The developers have decided to deprecate this style of loops from 2.2 onwards, and completely remove it 3.0 onwards. The rationale provided by Apple is that the usage of semicolons doesn't sit well with their style guide, and is "one more step towards never typing a semi-colon again." Now all loops are written using the range function, and in the case one needs to increment the loop counter by more than 1, the use of the stride(to:by:) is encouraged.

6. DEPRECATION OF ++ AND --:

A similar reasoning has been followed by Apple in deprecating the ++ and -- operators. Apart from the fact that it only steepened the learning curve for those unfamiliar with the operators, it also brought about a sense of confusion as to whether they are being used prefix or postfix. And considering that using `i += 1` is not at all more complicated than `i++`, not much has been lost with the deprecation of this C style programming construct.